
Java
spring®
Boot

What is Spring Boot?

Java Spring Boot is a tool made using the open-source Java Spring framework. It is primarily used to create web apps and microservices. It simplifies project setup with its auto-configuration features and easy startup with embedded servers.

Why use Spring Boot?

1

Faster development

Spring maintains minimal boilerplate code by using various features such as dependency injection and auto-configuration. Contains automatic dependency resolution and reduced need for repetitive code. It has default project structure setup.

2

Microservices-Ready

We can easily create small, focused services, which are independent and able to interact with other API. Offers fast development and scalability. Streamlined approach allows developers to focus on business logic

3

Standalone Apps

Can run the entire project as a Java app and contains embedded servers(Tomcat/Jetty) and provides starter dependencies reducing boilerplate code.

How it works

Your code:

@RestController classes

@Service components

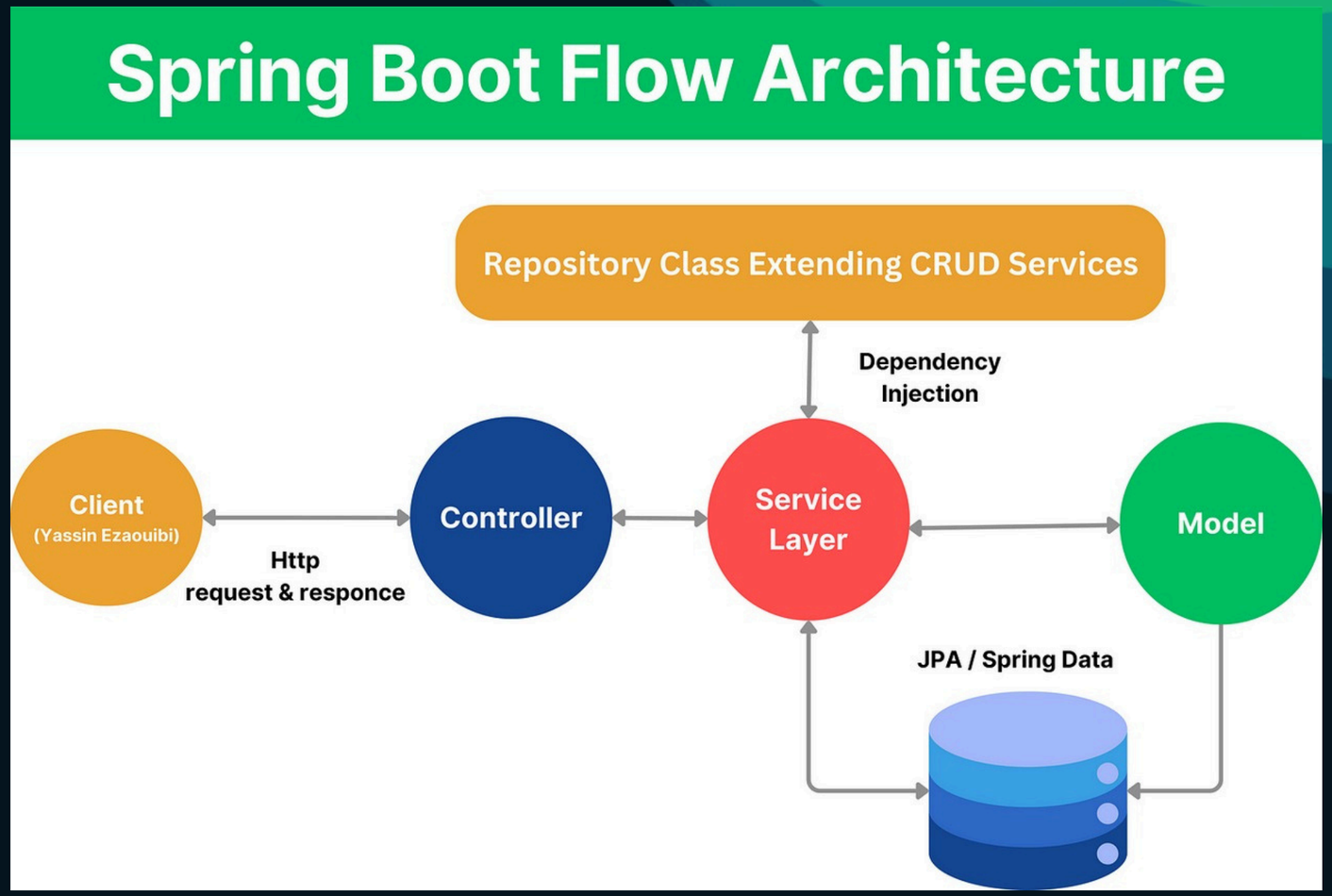
@Repository interfaces

Configuration:

Scans your classpath Applies 150+ default configurations Connects your Beans automatically Injects dependencies (@Autowired) Auto-starts on port 8080 (configurable)

Run the App:

Can run the app as an executable jar file. (java -jar your-app.jar) Provides access to all the REST endpoints given.



Architecture

Controllers

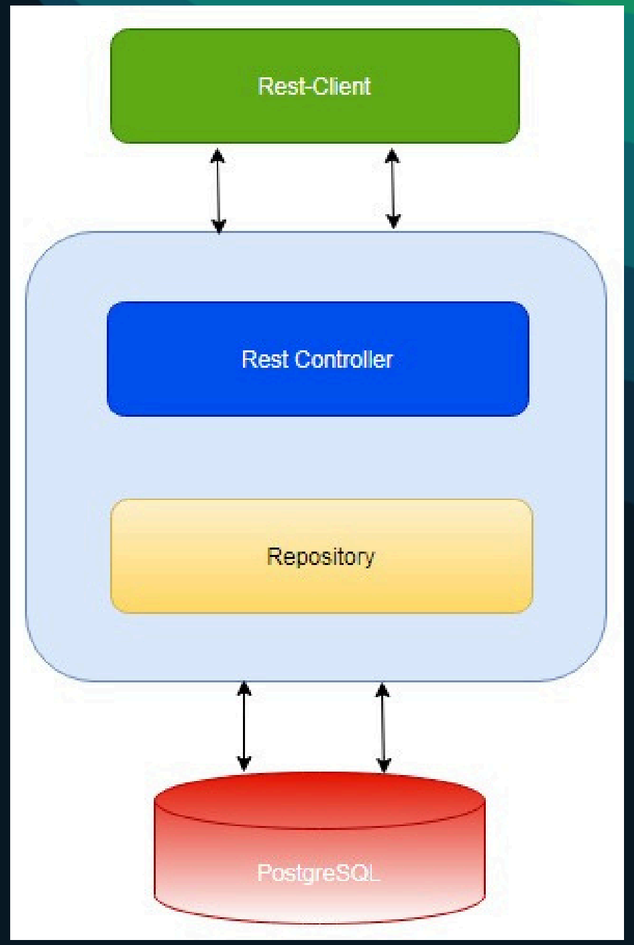
A Java class responsible for handling incoming HTTP requests and returning a response. Declared using @RestController and endpoints and handled using @RequestMapping

Service

Service layer is a Java class used to hold the business logic behind the controller endpoints. Declared using @Service.

Repository

The repository layer handles data access, providing CRUD operations and custom queries. For example, finding employee by their ID.



Spring Bean

The objects that form the backbone of your application and that are managed by the Spring IoC container are called beans. A bean is an object that is instantiated, assembled, and otherwise managed by a Spring IoC container.

An example of Service bean:

```
@Service
public class UserService {
@Autowired
    private UserRepository
userRepository; // Injected dependency
    public User findById(Long id) {
        return
userRepository.findById(id).orElseThro
w();
    }
}
```

Spring MVC

- Handles HTTP requests/responses (REST APIs or web apps). Uses annotations to map URLs to methods:
- @RestController: Combines @Controller +
- @ResponseBody. @GetMapping, @PostMapping, etc. for HTTP methods. @PathVariable: Extracts values from
- URLs (e.g., /users/ {id}).
-

```
@RestController
public class UserController {
@GetMapping("/users/{id}")
public User
getUser(@PathVariable Long id) {
    return userService.findById(id);
 }
}
```

Spring Data JPA

- JPA (Java Persistence API): Standard ORM (Object-Relational Mapping).
- Spring Data JPA: Adds magic (reduces boilerplate code).
- Key Annotations:
- @Entity: Maps Java class to a database table.
- @Repository: Extends JpaRepository for CRUD operations.
- @Query: Custom SQL/JPQL queries.

```
@Repository
public interface UserRepository
extends JpaRepository<User, Long>
{
  User findByName(String name); //
Auto-implemented!
}
```

Spring Security

- Built-in login/logout flows.
- CSRF protection, password encoding.
- OAuth2 support (for social login).

Authentication (AuthN): Who are you? (e.g., username/password).
Authorization (AuthZ): What can you do? (e.g., admin vs user).

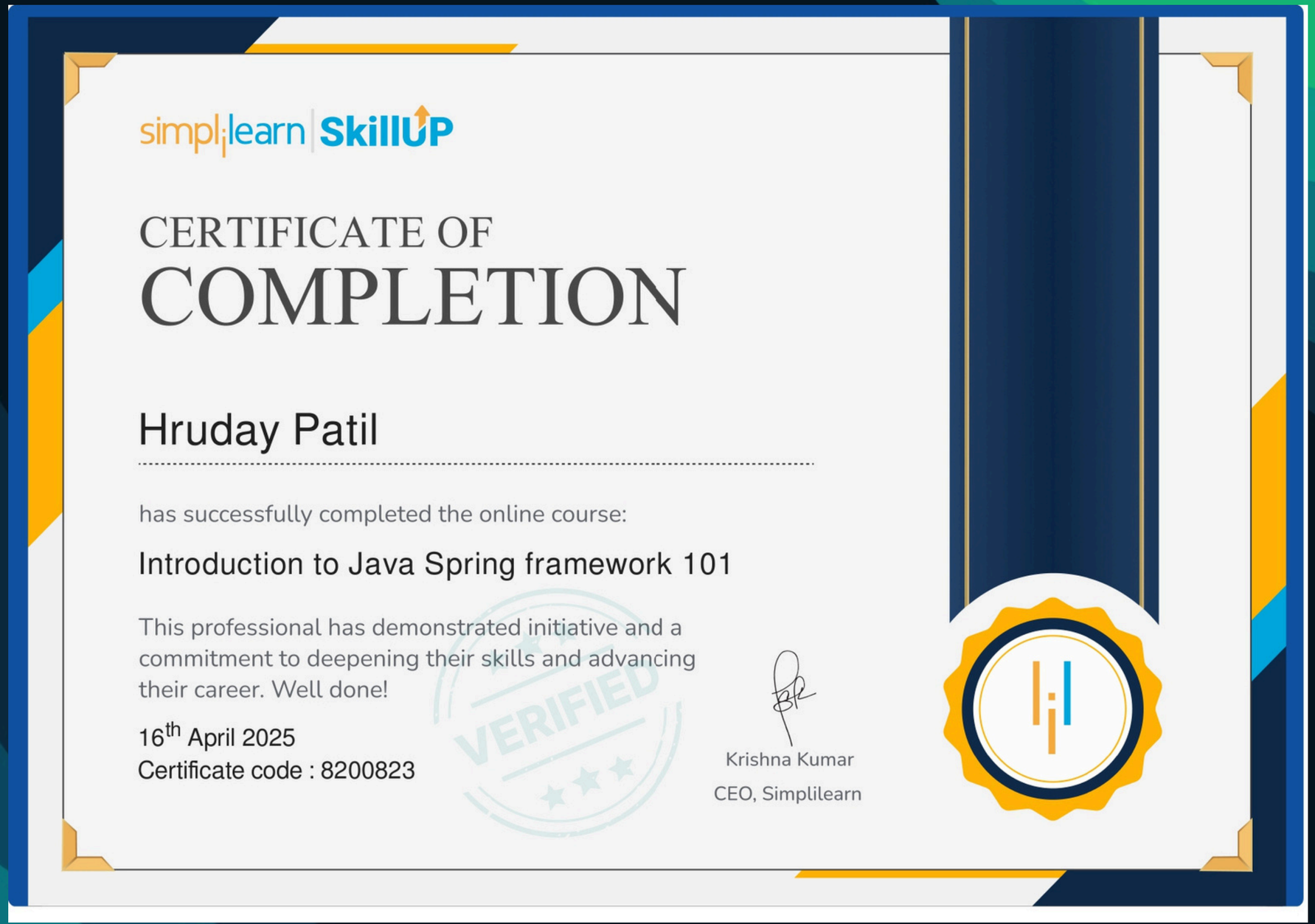
```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
@Bean
public SecurityFilterChain
securityFilterChain(HttpSecurity
http) throws Exception {
}
}
```

Conclusion

Popular use-cases of Java Spring are Netflix & LinkedIn

Ideal for users:

- REST APIs (backend services).
- Microservices (cloud-native apps).
- Batch processing (scheduled jobs).
- Prototyping


simplilearn SkillUP
CERTIFICATE OF
COMPLETION
Hruday Patil
has successfully completed the online course:
Introduction to Java Spring framework 101
This professional has demonstrated initiative and a commitment to deepening their skills and advancing their career. Well done!
16th April 2025
Certificate code : 8200823
VERIFIED
Krishna Kumar
CEO, Simplilearn

Thank You